

Adversarial Monte Carlo Tree Search with Stratified Sampling for Competitive 8-Ball Pool

AI Collaborative Development Group

Abstract—Designing competitive agents for 8-ball pool requires balancing aggressive offensive play with robust defensive strategy under stochastic physical dynamics. This paper details the development of **NewAgentPro**, an agent that underwent a transition from heuristic-based sampling to a pure adversarial Monte Carlo Tree Search (MCTS) framework. We address the limitations of continuous action space search through the implementation of progressive widening, RAVE, and stratified sampling. Furthermore, we introduce an opponent-aware reward function with an adaptive weighting scheme (the “Lambda Ladder”) and a risk-aware double verification mechanism. Experimental results demonstrate that **NewAgentPro** achieves a competitive win rate of 45.2% against the high-performance **BasicAgentPro** baseline, while significantly reducing turnovers through prioritized foul control.

I. INTRODUCTION

Autonomous decision-making in complex physical environments remains a cornerstone of artificial intelligence, with applications spanning robotics, industrial automation, and competitive gaming. Billiards, and specifically 8-ball pool, provides a uniquely challenging testbed for these algorithms. Unlike discrete board games like Chess or Go, billiards features a high-dimensional continuous action space, intricate collision dynamics governed by friction and spin, and significant environmental noise.

This paper details the development of **NewAgentPro**, a competitive AI agent designed to operate in the high-stakes environment of adversarial pool. The central challenge addressed here is the design of a search-driven architecture capable of competing with state-of-the-art baselines like **BasicAgentPro**, which employs sophisticated lookahead search. Traditional heuristic-based agents, while efficient, often fail to account for the long-term strategic consequences of a shot, such as leaving an easy clearance for the opponent or triggering a catastrophic foul due to execution noise.

We propose an adversarial Monte Carlo Tree Search (MCTS) framework tailored for the continuous and stochastic nature of pool. Our approach emphasizes the transition from hand-crafted heuristic scoring to pure physics-based evaluation, supported by advanced tree search techniques such as Progressive Widening and Rapid Action Value Estimation (RAVE). Furthermore, we introduce a suite of engineering optimizations—stratified sampling, adaptive risk weighting (the “Lambda Ladder”), and multi-stage verification—to bridge the gap between theoretical search and real-world execution.

*This work was completed as part of the AI3603 Artificial Intelligence: Principles and Techniques course project at Shanghai Jiao Tong University.

II. RELATED WORK

A. Continuous Action Space Search

Monte Carlo Tree Search has revolutionized sequential decision-making, particularly after its success in Go [1]. However, standard MCTS is designed for discrete action spaces. In continuous domains like billiards, the branching factor is infinite. Techniques like Progressive Widening [2] and Double Progressive Widening have been developed to manage this by limiting the number of children explored based on the node’s visit count, allowing the search to gradually broaden its horizon.

B. Evolutionary Strategies vs. Tree Search

In the context of this course project, another prominent approach is the combination of geometric heuristics (Ghost Ball) with local refinement using Evolution Strategies like CMA-ES [3]. While Evolution Strategies are highly efficient at local optimization within a small action volume, they often lack the global strategic depth inherent in MCTS. Our work explicitly contrasts these paradigms, demonstrating how a search-driven approach can naturally incorporate adversarial reasoning and sequential planning where local refinement may fall into strategic traps.

C. Adversarial Planning in Stochastic Games

Competitive play requires reasoning about the opponent’s likely response. Minimax-style search is difficult in stochastic environments. MCTS handles this through the selection and backpropagation logic, where the reward function can be augmented with opponent-aware terms. Previous work has explored various ways to model opponent threat, ranging from simple proximity heuristics to deep value networks.

III. PROBLEM FORMULATION

We consider the 8-ball pool game as a stochastic Markov Decision Process (MDP) with a continuous action space $\mathcal{A} \subset \mathbb{R}^5$, representing shot parameters $(V_0, \phi, \theta, a, b)$. Each action is subject to Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$. The agent’s goal is to maximize the probability of winning, which involves not only pocketing own-group balls but also mitigating opponent threats.

We define an adversarial reward function at each state s :

$$R(s) = R_{\text{own}}(s) - \lambda \cdot \text{max_threat}_{\text{opp}}(s) \quad (1)$$

where $R_{\text{own}}(s)$ is the immediate potting reward, and $\text{max_threat}_{\text{opp}}(s)$ is a heuristic estimate of the opponent’s best possible response. The parameter λ controls the defensive sensitivity.

IV. METHODOLOGY

Our agent architecture follows a multi-stage “search-and-verify” pipeline designed to maximize strategic depth while maintaining execution robustness.

A. Adversarial MCTS Framework

The core decision engine is an MCTS that operates on an action pool $\mathcal{A}_{cand}$ generated at each state. Unlike standard MCTS that samples a single action per expansion, we pre-generate a diverse set of candidates and use Progressive Widening to control their entry into the tree.

The selection of a node is governed by the Upper Confidence Bound for Trees (UCT), modified for the adversarial setting:

$$UCT(v, i) = \frac{Q(v, i)}{n(v, i)} + C \sqrt{\frac{\ln N(v)}{n(v, i)}} \quad (2)$$

where Q is the accumulated reward, n is the visit count of child i , and N is the visit count of the parent.

B. RAVE: Rapid Action Value Estimation

To address the sparsity of rewards and accelerate convergence in the high-dimensional action space, we integrate ****RAVE****. In billiards, many actions targeting the same ball share similar physical outcomes even if their precise spin parameters differ. RAVE exploits this property by maintaining a value $Q_{RAVE}(T)$ for each target ball T . This value is updated using any simulation where a shot targeting T was performed. The final value used for selection is a weighted combination:

$$Q_{final} = (1 - \beta)Q_{UCT} + \beta Q_{RAVE} \quad (3)$$

where β is a dynamic weight that decreases as the visit count n increases. This allows RAVE to provide a robust prior that guides the search in the early stages, while UCT provides high-precision evaluation as more rollouts are collected.

C. Stratified Action Sampling and Truncation Bias

A common failure mode in MCTS for billiards is the concentration of samples on a few nearby balls, missing high-quality shots on distant targets. This “truncation bias” occurs when the candidate generation stage prioritizes balls by simple proximity.

We implement a ****Stratified Sampling**** scheme (Algorithm 1). By grouping potential actions by their target balls and ensuring a uniform distribution of candidates (up to 6 unique targets, with multiple velocity and spin variants per target), we ensure the search explores the entire table. In V4.x, the candidate count K was increased to 64, significantly improving the agent’s ability to find complex clearance paths.

Algorithm 1 Stratified Action Enumeration

Require: Table state s , Candidate count K

Ensure: Action set $\mathcal{A}_{cand}$

```

1:  $G \leftarrow \text{GroupActionsByTarget}(s)$     ▷ Grouped by target ball ID
2:  $\mathcal{A}_{cand} \leftarrow \emptyset$ 
3:  $targets \leftarrow \text{SortTargetsByDistance}(s)$ 
4:  $i \leftarrow 0$ 
5: while  $|\mathcal{A}_{cand}| < K$  and  $targets$  is not empty do
6:    $target \leftarrow targets[i \text{ (mod } |targets|)]$ 
7:   if  $G[target]$  is not empty then
8:      $a \leftarrow \text{PopRandomAction}(G[target])$ 
9:      $\mathcal{A}_{cand} \leftarrow \mathcal{A}_{cand} \cup \{a\}$ 
10:  end if
11:   $i \leftarrow i + 1$ 
12: end while
13: return  $\mathcal{A}_{cand}$ 

```

D. Opponent-Aware Reward and the Lambda Ladder

The weight λ in Equation 1 is critical for balancing aggression and defense. A fixed λ often leads to either excessive risk-taking or paralyzing conservatism. We propose the **Lambda Ladder**, an adaptive scheduling mechanism where λ is a function of the game stage and remaining balls N_{rem} :

$$\lambda(N_{rem}) = \begin{cases} 0.02 & \text{if } N_{rem} > 3 \text{ (Aggressive)} \\ 0.18 & \text{if } N_{rem} \leq 3 \text{ (Soft Defense)} \\ 0.00 & \text{if targeting 8-ball (Finalize)} \end{cases} \quad (4)$$

This allows the agent to play aggressively in the early game to clear the table and switch to a defensive posture in the endgame, while removing defensive hesitation when attempting the winning shot.

E. Foul Priority Filtering

To ensure robust play, we implement a multi-layered filtering system during the candidate selection stage. Actions are evaluated and filtered according to a strict priority hierarchy based on simulated statistics:

- 1) **cat_rate**: Filtering out actions with any probability of catastrophic fouls (e.g., cue ball scratch or illegal 8-ball contact).
- 2) **nohit_rate**: Prioritizing actions that guarantee legal first contact.
- 3) **norail_rate**: Penalizing failure to hit a cushion after contact (no-rail foul).

This hierarchy ensures that the agent first identifies “legal” actions before optimizing for score.

V. MCTS VS. EVOLUTION STRATEGIES (CMA-ES)

A critical comparison in this project is between our MCTS framework and the Hybrid Heuristic-Evolution (HHE) method (e.g., Ghost Ball + CMA-ES). These two paradigms represent distinct philosophies in billiards AI.

A. Precision vs. Foresight

The HHE method excels at high-precision local optimization. By using a Ghost Ball geometric prior, it identifies a physically plausible shot and uses CMA-ES to refine parameters like spin and velocity. This results in extremely high potting accuracy for single shots. However, HHE is inherently myopic; its reward function typically considers only the immediate state after the shot.

In contrast, *NewAgentPro*’s MCTS naturally incorporates lookahead. While MCTS may not achieve the same sub-millimeter precision as CMA-ES (due to the broader search across the action pool), it can identify sequences of play. For instance, MCTS can choose a slightly harder pot that leaves the cue ball in a position that blocks the opponent, a decision that a myopic HHE agent would likely miss.

B. Robustness to Non-Convexity

The reward landscape in pool is highly non-convex and discontinuous (pot-or-fail). CMA-ES, as a local optimizer, can get stuck in local optima if the initial geometric prior is slightly off or if spin effects create a non-linear trajectory. MCTS, by sampling across a diverse action pool and maintaining a search tree, is more robust to these discontinuities, as it can explore multiple distinct regions of the action space simultaneously.

C. Computational Trade-offs

HHE is significantly more computationally efficient. A typical CMA-ES refinement might take 16-32 simulations per shot. An effective MCTS search for *NewAgentPro* requires 160-240 simulations to achieve strategic parity. This highlights the “performance-cost” trade-off: MCTS buys strategic stability at the price of higher wall-clock game time.

VI. IMPLEMENTATION CHALLENGES

A. Noise Realignment and Double Verification

We observed a significant performance gap between MCTS simulations and the actual environment, termed “execution hallucinations.” Analysis revealed that the internal noise model used for search did not match the environment’s physics. By explicitly synchronizing the standard deviations $\sigma_\phi = 0.1$ and $\sigma_{spin} = 0.003$ with the `PoolEnv` parameters, we reduced the cue ball scratch rate by approximately 15% and significantly improved search reliability.

To further mitigate the impact of rollout stochasticity, we introduce a ****Double Verification**** mechanism. For the action with the highest MCTS score, we perform an additional 40 pressure simulations.

$$\text{ActionSafe} = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \mathbb{I}[\text{foul}_i = 0] > 0.98 \quad (5)$$

If the optimal action fails this verification ($N_{test} = 40$), the agent automatically falls back to a safer sub-optimal action, ensuring high tolerance for error in critical shots.

B. Technical Bottlenecks and Solutions

Table I summarizes the key technical challenges encountered during development and the corresponding engineering solutions, which were instrumental in reaching the final performance milestones.

TABLE I
TECHNICAL BOTTLENECKS AND SOLUTIONS

Challenge	Root Cause	Solution
Low win rate for Player B	Insufficient FOUL_NO_RAIL penalty	Elevated rail foul to catastrophic level; added MCTS filtering layer.
8-ball hesitation	$\lambda > 0$ during endgame	Set $\lambda = 0$ for 8-ball shots.
Long-range potting failure	V_0 limit too restrictive (5.2 m/s)	Increased V_0 limit to 6.2 m/s; added velocity variants.
Break shot scratch	High density of ball rack	Specialized break strategy with narrowed angle perturbations.

VII. ALGORITHM EVOLUTION

The development of *NewAgentPro* followed a rigorous version roadmap, with each major iteration addressing a fundamental limitation discovered during tournament evaluation.

A. V1.x: Heuristic-Based Sampling

The initial version was based on the *OptimizedNewAgent* framework. It utilized Bayesian optimization to tune the sampling density of a Ghost Ball-inspired heuristic.

- **Method:** Focused on immediate potting probability using simple geometric lines.
- **Limitation:** Suffered from extreme myopia. It could not understand that a difficult pot might be strategically superior if it left the opponent in a difficult position.
- **Performance:** Failed consistently against *BasicAgentPro* due to lack of lookahead.

B. V2.x: Transition to Pure MCTS

In the second stage, we replaced the heuristic scoring with an MCTS framework driven by ****pure physics simulation results****.

- **Method:** Introduced Progressive Widening and RAVE.
- **Major Shift:** Rewards were no longer hand-crafted; instead, the agent optimized for the expected state value calculated through rollout trajectories.
- **Breakthrough:** This version significantly improved the agent’s ability to clear open tables, as the search naturally identified sequences of shots.

C. V3.x: Adversarial Modeling and fixed Lambda

The third stage focused on opponent-aware decision-making. We introduced the adversarial reward $R = R_{own} - \lambda R_{opp}$.

- **Method:** Initialized λ at values between 0.35 and 0.75.

The comparison between MCTS and local refinement strategies (CMA-ES) further underscores that while local optimization is superior for single-shot precision, global tree search is required for the strategic foresight necessary in tournament-style play.

X. FUTURE WORK

Despite its competitive performance, `NewAgentPro` leaves several avenues for future exploration:

- **Deep Value Networks:** Replacing the current hand-crafted reward function and opponent threat heuristics with a deep neural network trained on millions of simulated games could further improve strategic depth and search efficiency.
- **Dynamic Action Generation:** Currently, candidates are pre-generated. A more integrated approach where the tree search itself guides the sampling of new continuous actions could lead to better precision without the computational overhead of exhaustive sampling.
- **Opponent Modeling:** Future versions could explicitly learn the playstyle of the opponent (e.g., their tendency to over-cut balls or fail at cushions) and adapt the adversarial weights in real-time.

DIVISION OF LABOR

AI Collaborative Development Group:

- *Algorithm and Engineering:* Designed the MCTS framework, implemented stratified sampling, and developed the noise-alignment and verification mechanisms.
- *Experimental Analysis:* Conducted the tournament evaluations, performed the ablation studies on strategic weighting, and maintained the reproducibility of the codebase.
- *Reporting:* Drafted the manuscript, formalized the algorithm descriptions, and visualized the experimental results.

ACKNOWLEDGMENT

The authors thank the AI3603 course staff at Shanghai Jiao Tong University for providing the simulation environment and baseline agents. This report was collaboratively prepared by the development group as part of the course requirements.

REFERENCES

- [1] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [2] R. Coulom, “Efficient selectivity and backup operators in monte-carlo tree search,” in *Computers and Games*, ser. Lecture Notes in Computer Science, H. J. van den Herik, P. Ciancarini, and H. H. L. M. Donkers, Eds., vol. 4630. Springer, 2007, pp. 72–83.
- [3] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.